

XIAMEN UNIVERSITY MALAYSIA
厦門大學馬來西亞分校

并查集

Union-Find: Disjoint Set Union Data Structure

作者：Wu Youfan
School of Computing and Data Science, Xiamen University Malaysia
2026 年 10 月 30 日

目录

1. 并查集简介
2. 朴素并查集
3. 路径压缩
4. 按秩合并
5. 复杂度分析
6. 例题：最小生成树

目录

⑦ 总结

目录

1 并查集简介

2 朴素并查集

3 路径压缩

4 按秩合并

5 复杂度分析

6 例题：最小生成树

目录

7 总结

什么是并查集

并查集：一种专门用于管理**不相交集合**的数据结构。

通俗理解

- 想象一群人站队，每人只记得自己的“父亲”是谁
- 想判断两个人是否同队，就沿父亲一路往上找，看最终是否走到同一个人
- 父亲链的顶端就是这队的“代表”，也叫根

典型问题

- 动态加边，随时询问两个点是否连通
- 统计当前有多少个互不相交的连通块

并查集 = 用树管理集合，根代表集合

并查集支持的操作

两个核心操作

- 查询 find(x)：找到 x 所在集合的根
- 合并 unite(x, y)：把 x 和 y 所在的两个集合合并成一个

两个操作的关系

- find 是 union 的基础：先找到两个根，再决定谁挂到谁下面
- 若 find 太快，unite 也跟着快
- 优化的重点，几乎都在 find 上

常见变体

- 带权并查集：节点上存有到父节点的权值
- 种类并查集：拆点表示“朋友 / 敌人”等关系

find 一次，找根；unite 一次，合并两根

目录

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

1. 并查集简介
2. 朴素并查集
3. 路径压缩
4. 按秩合并
5. 复杂度分析
6. 例题：最小生成树

目录

⑦ 总结

例题：朋友圈

题目：n 个人，m 对朋友关系。朋友的朋友也算朋友，问最终一共有多少个朋友圈。

示例

- $n = 5$，关系 $(1, 2)$、$(3, 4)$，答案为 3
- $n = 4$，关系 $(1, 2)$、$(2, 3)$、$(3, 4)$，答案为 1
- $n = 3$，没有任何关系，答案为 3

朋友圈：解法

核心思路：把每个人看成一个集合，每合并一对朋友就少一个圈，最后数根的数量。

关键步骤

- 第一步：初始 $fa[i] = i$，每个人自成一个圈
- 第二步：对每对朋友 (u, v)，先 $find(u)$、$find(v)$
- 第三步：若两个根不同，把其中一个根挂到另一个根下面，圈数减一
- 第四步：处理完所有关系后，统计满足 $fa[i] = i$ 的节点数

为什么正确：每次合并把两个圈并成一个，圈的数量恰好减一；最终每个圈只剩一个根，数根就是数圈。

朴素版本复杂度：单次操作最坏 $O(n)$，总 $O(nm)$。

朋友圈：朴素代码


```
int fa[MAXN];
void init(int n) {
    for (int i = 1; i <= n; ++i) fa[i] = i;
}
int find(int x) {
    while (fa[x] != x) x = fa[x];    // 沿父指针找根
    return x;
}
void unite(int x, int y) {
    int rx = find(x), ry = find(y);
    if (rx != ry) fa[rx] = ry;       // 把一棵树的根挂到另一棵
}
int count_blocks(int n) {
    int cnt = 0;
    for (int i = 1; i <= n; ++i)
        if (fa[i] == i) ++cnt;
    return cnt;
}
```

目录

1 并查集简介

2 朴素并查集

3 路径压缩

4 按秩合并

5 复杂度分析

6 例题：最小生成树

目录

7 总结

朴素 find 的问题

问题场景：连续合并 $1 \to 2 \to 3 \to 4 \to \cdots \to n$，父亲链变成一条长链。

后果

- $find(1)$ 需要走 n 步
- 若之后还要查询 1，每次都重新走 n 步
- 总复杂度退化为 $O(nm)$，数据稍大就超时

思考方向

- 父亲链上的所有节点，其实都同属一个集合
- 不妨把它们直接挂到根下面，一次查询后整条链变短

链太长是瓶颈，扁平化是关键

路径压缩：思路

核心想法：在 find 的过程中，把路径上的每个节点都直接连到根。

做法

- 递归找根，回溯时把当前节点的父亲改成根
- 一次 find 之后，从 x 到根的整条链变成两段：x 直接连根

效果

- 第一次查询虽然还走了长链，但把链压扁了
- 之后查询 x 或链上任意节点都是 $O(1)$
- 均摊复杂度接近常数

注意：路径压缩只优化 find，不改 unite 的结构。

边查边改父亲，一次压扁整条链

路径压缩代码


```
1 int fa[MAXN];
2 int find(int x) {
3     if (fa[x] == x) return x;
4     return fa[x] = find(fa[x]);     // 回溯时把父亲改成根
5 }
```

目录

1 并查集简介

2 朴素并查集

3 路径压缩

4 按秩合并

5 复杂度分析

6 例题：最小生成树



目录

7 总结

朴素 unite 的问题

问题场景：不关心两棵树的相对大小，直接把前者挂到后者，可能把大树挂到小树。

后果

- 树高不断变大，find 也要走更多步
- 极端情况下退化成一条长链，与不做优化等价
- 即使有路径压缩，第一次查询仍然偏慢

思考方向

- 让较小的树挂到较大的树下面
- 树高增长慢，find 走的步数就少

小树挂大树，树高才增长得慢

按秩合并：思路

核心想法：合并时比较两棵树的高度（秩），矮的挂到高的下面。

做法

- 维护 $rk[u]$：以 u 为根的树的高度
- 合并时先找两个根，比较 rk：矮的挂到高的下面
- 若高度相同，随便挂，把新的根秩加 1

效果

- 树高始终不超过 $O(\log n)$
- 单独使用按秩合并，find 与 unite 都是 $O(\log n)$
- 与路径压缩叠加，复杂度接近常数

两种优化：路径压缩 + 按秩合并，是并查集的标配组合。

先比秩，再挂树

按秩合并代码


```
int fa[MAXN], rk[MAXN];
void init(int n) {
    for (int i = 1; i <= n; ++i) { fa[i] = i; rk[i] = 1; }
}
int find(int x) {
    return fa[x] == x ? x : fa[x] = find(fa[x]);
}
void unite(int x, int y) {
    int rx = find(x), ry = find(y);
    if (rx == ry) return;
    if (rk[rx] < rk[ry]) swap(rx, ry);   // 保证 rx 秩不小于 ry
    fa[ry] = rx;
    if (rk[rx] == rk[ry]) ++rk[rx];      // 等高时新的根秩加 1
}
```

目录

1 并查集简介

2 朴素并查集

3 路径压缩

4 按秩合并

5 复杂度分析

6 例题：最小生成树

目录

7 总结

三种实现的复杂度对比

常见实现与复杂度

实现方式	find 复杂度	unite 复杂度
朴素	$O(n)$	$O(n)$
仅路径压缩	$O(\log n)$ 均摊	$O(\log n)$ 均摊
路径压缩 + 按秩合并	$O(\alpha(n))$	$O(\alpha(n))$

反阿克曼函数 $\alpha(n)$

- 增长极慢，$\alpha(10^{18}) \approx 4$
- 实际应用中几乎视为常数
- 是“接近 $O(1)$ 但不是严格 $O(1)$”的经典例子

路径压缩 + 按秩合并，几乎可以视为常数时间

目录

1 并查集简介

2 朴素并查集

3 路径压缩

4 按秩合并

5 复杂度分析

6 例题：最小生成树

目录

7 总结

例题：最小生成树

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

题目：给定 n 个点、m 条边的带权无向图，求最小生成树的总权值。若图不连通，输出 -1。

示例

- $n = 4$，边 $(1, 2, 1)$、$(2, 3, 2)$、$(3, 4, 3)$，答案为 6
- $n = 3$，边 $(1, 2, 5)$、$(2, 3, 2)$、$(1, 3, 3)$，答案为 5
- $n = 3$，边 $(1, 2, 1)$，答案为 -1（图不连通）

最小生成树：解法

核心思路：按边权从小到大逐边尝试，只要两端还没连通就加入这条边，直到凑满 $n-1$ 条。

关键步骤

- 第一步：把所有边按权值从小到大排序
- 第二步：初始化并查集，每个点自成一个分量
- 第三步：依次考察每条边 (u, v, w)，用 find 判断两端是否同分量
- 第四步：不同分量就合并两端，把 w 加入答案，边数加一
- 第五步：边数达到 $n-1$ 时提前结束；否则图不连通，输出 -1

为什么正确：最小生成树一定包含当前权值最小的、不构成环的边；并查集恰好负责“不构成环”的判断，两端同分量就说明加进去会成环。

复杂度：排序 $O(m \log m)$，加边 $O(m\alpha(n))$。

最小生成树代码


```cpp
struct Edge {
    int u, v, w;
    bool operator<(const Edge& o) const { return w < o.w; }
};
int kruskal(int n, vector<Edge>& edges) {
    init(n);
    sort(edges.begin(), edges.end());
    int ans = 0, cnt = 0;
    for (auto& e : edges) {
        int ru = find(e.u), rv = find(e.v);
        if (ru != rv) {
            fa[ru] = rv;
            ans += e.w;
            ++cnt;
            if (cnt == n - 1) break;
        }
    }
    return cnt == n - 1 ? ans : -1;
```

最小生成树代码


```
19 }
```

目录

1 并查集简介

2 朴素并查集

3 路径压缩

4 按秩合并

5 复杂度分析

6 例题：最小生成树

目录

⑦ 总结

总结（一）：并查集核心

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

三句话记住并查集

- 一个数组 fa，把自己所在的集合组织成树
- 一个函数 find，沿父指针找到根
- 一个函数 unite，把两个根合并

两个关键细节

- 初始化 $fa[i] = i$，$rk[i] = 1$
- find 里一定带路径压缩，别写成空函数

find 找根，unite 合并根

总结（二）：两种优化

优化	解决问题	效果
路径压缩	find 走得太远	把路径压扁，均摊接近常数
按秩合并	unite 树高失控	树高 $O(\log n)$

易错点

- 忘记初始化 $fa[i] = i$，根找错了
- find 里没写路径压缩，复杂度退化
- 合并时先比较秩再挂，不要直接 $fa[rx] = ry$
- 递归写 find 在超深链上可能爆栈，可以改迭代版

两个优化叠加，复杂度几乎为常数

总结（三）：并查集的常见应用

四类典型场景

- 连通性判定：动态加边，随时问两点是否连通
- 连通块计数：合并完数一下根的数量
- 最小生成树：Kruskal 中判断是否成环
- 带权问题：种类并查集、带权并查集

进一步学习方向

- 带权并查集：维护到根的差值
- 种类并查集：拆点表示多种关系
- 可持久化并查集：回到历史版本查询

并查集是图论与动态连通问题的万金油

附录：带权并查集简介

场景：在维护连通性的同时，还要记录每个节点到根的某种权值（例如距离、差值）。

思路

- 额外维护 $dis[x]$：x 到其父亲的边权
- find 时先递归求根，回溯时把 $dis[x]$ 累加到根的路径上
- 查询 x 到 y 的权值差就是 $dis[x] - dis[y]$

要点

- 路径压缩要把 dis 一起更新，不能只改 fa
- 合并时还要设置新的 $dis[ry]$ 为对应权值差
- 常用于食物链、种类关系、区间差分等问题

学习建议：先把普通并查集吃透，再动手写带权版本，代码顺序要格外小心。

路径压缩与权值更新，必须同步进行

附录：常见陷阱

初始化

- 忘记初始化 $fa[i] = i$，导致一开始就找不到根
- 数组下标从 1 还是从 0 开始，全文要统一

find

- 只写递归不压缩，复杂度会退化
- 超深链上递归可能爆栈，可以改迭代写法

unite

- 直接 $fa[x] = y$，把节点挂到节点下面，破坏了并查集的结构
- 正确做法：先 find 找到两个根，再把一个根挂到另一个根下面

Kruskal 特有

- 忘记按边权排序，答案错误
- 判断是否连通应检查加入了 $n - 1$ 条边